



WALL-E: AUTONOMOUS SURVEILLANCE ROBOT

Personal Project



Contents

Abstract	2
Introduction.....	2
Robot System Design (hardware, software architecture)	2
Control Algorithms	5
Testing and Results	6
Lessons Learned.....	8
Future Enhancements.....	9
Attach diagrams, flowcharts, and photos.....	10

Abstract

This project presents an autonomous obstacle-avoiding surveillance robot built using a Raspberry Pi as the main controller. The robot employs an HC-SR04 ultrasonic sensor mounted on a servo motor to scan the surrounding environment, measure distances to obstacles, and select the safest direction of movement. Two DC motors driven by an L298N motor driver provide differential-drive motion, while pulse-based PWM control and soft-start techniques ensure smooth and stable operation. An RGB LED and a buzzer are used to indicate the system's operational status. Additionally, a Raspberry Pi camera captures images whenever a critical obstacle is detected, and alert messages with the captured photos are sent to the user via the Telegram platform, enabling remote monitoring of the robot's environment. The system demonstrates key concepts of mobile robotics, including real-time sensing, decision-making, motor control, and wireless communication.

Introduction

With the advantages that they can move freely on ground without being monitored by human, autonomous mobile robots are becoming more and more prevalent in applications such as safety monitoring, inspection and surveillance. These robots must be endowed with the appropriate sensors, actuators and intelligent algorithms to enable them to perceive, act and navigate in a safe way. The most important capability needed for a mobile robot to work in dynamic or unknown environment is obstacle detection and avoidance.

Aim of the project : The main objective of the project is to build an autonomous surveillance bot on which features like road traffic collision avoidance, Object detection and Real time observation are included. Ultrasonic ranging, infrastructure scanning using servos, motor control and wireless communication all involve applied aspects of mobile robotics and embedded systems.

The robot is programmed in Python, using the gpiozero library to control hardware components such as sensors, motors and lights. The capabilities of the robot are to send alert messages and take pictures on Telegram platform, rather than self localization for remote monitoring. The project offers hands-on experience in autonomy, real-time decision making, and system integration of the robotic equipment.

Robot System Design (hardware, software architecture)

Robot System Design

The robot system is designed using a combination of **hardware components** and **software control architecture** to achieve autonomous obstacle avoidance and remote monitoring. The Raspberry Pi acts as the central controller that connects all sensors, actuators, and communication modules.

1. Hardware Design

The hardware of the robot consists of the following main components:

1.1 Processing Unit

- **Raspberry Pi**
The Raspberry Pi is the brain of the robot. It processes sensor data, makes decisions, and controls motors, indicators, and communication. It runs the Python program that manages all robot operations.

1.2 Power Supply

- **Three Lithium Batteries (18650 cells)**
The robot is powered using three lithium batteries, which provide sufficient voltage and current for the motors, Raspberry Pi, sensors, and peripherals. This makes the robot portable and suitable for autonomous operation.

1.3 Sensor System

- **HC-SR04 Ultrasonic Sensor**
Used to measure the distance between the robot and obstacles.
- **Servo Motor**
The ultrasonic sensor is mounted on a servo motor, allowing it to rotate from left to right to scan the surroundings and find the safest direction.

1.4 Actuators

- **DC Motors (2 units)**
Used for robot movement.
- **L298N Motor Driver Module**
Controls motor direction and speed. It allows the robot to move forward, turn left, turn right, and stop.

1.5 Output and Feedback Devices

- **RGB LED**
Indicates robot status such as normal movement, obstacle detection, and turning.
- **Buzzer**
Provides an audio alert when an obstacle is detected.

1.6 Camera and Communication

- **Raspberry Pi Camera Module**
Captures images when an obstacle is detected.

- **Telegram** **Communication**
Sends messages and photos to the user for remote monitoring.

1.7 Mechanical Structure

- **Wooden** **Chassis**
Supports all components in multiple layers.
- **Wheels** **and** **Castor** **Wheel**
Provide stability and smooth movement.

2. Software Architecture

The software is written in **Python** and runs on the Raspberry Pi. It follows a simple and continuous control loop to manage sensing, decision-making, and movement.

2.1 Sensor Data Processing

- The ultrasonic sensor continuously measures distance.
- Multiple readings are taken and filtered using a **median filter** to reduce noise and improve accuracy.

2.2 Decision-Making Logic

- If the detected distance is safe, the robot moves forward.
- If an obstacle is detected within a set limit:
 - The robot stops
 - An alert is triggered
 - The servo rotates the ultrasonic sensor to scan left and right
 - The direction with the maximum distance is selected

2.3 Motor Control

- Motor speed is controlled using **PWM (Pulse Width Modulation)**.
- A **soft-start mechanism** is used to avoid sudden motor jerks.
- Pulse-based movement is used for smooth and controlled motion.

2.4 Alert and Monitoring System

- The buzzer sounds when an obstacle is detected.
- The RGB LED changes color based on robot state.

- The camera captures an image of the obstacle.
- The image and alert message are sent to the user through **Telegram**.

2.5 Main Control Loop

- The program runs in an infinite loop.
- It continuously checks distance, updates movement, and reacts to obstacles in real time.
- The system stops safely when interrupted.

Control Algorithms

The robot created here uses different kinds of algorithms implemented on a raspberry pi 5 using python, the control system manages the obstacle avoidance and surveillance tasks by processing sensor data and generating immediate motor responses that are necessary. The hardware components that were mentioned above in the robot system design section are all controlled using the gpiozero python library.

Motor Control:

Motor control is achieved using two DC motors driven by the L298N motor driver, the speed and direction are regulated using PWM based pulsed motion rather than a continuous signal, this approach was used because it enables a smoother low speed movement improved turning accuracy and reduced stress during startup, also a soft start mechanism is applied to prevent sudden jumps and losing its balance when the robot begins its moving.

Obstacle detection and avoidance:

It is performed using the HC-SR04 ultrasonic sensor that is mounted on a servo motor, that allows environmental scanning across a wide angular range rather than a fixed range, the distance measurements are taken across multiple angles, and a median filtering technique is applied to reduce the noise and improve the reliability of reading.

Five readings are collected, sorted and the median value is taken (selected), which effectively rejects the outlier measurements that may be caused by reflections or environmental noise.

Based on the predefined distance zones the robot will react accordingly to the measured proximity of it to the obstacles.

The distance zones table:

Zone	Distance threshold	Robot response
Critical	<65 cm	Scan -> take photo -> turn -> sidestep

Slow	65 – 145 cm	Caution (reduced speed)
Far	>145 cm	Normal (go forward)

When an obstacle is sensed in the critical zone, the robot will scan the environment around it, select the direction with the greatest clear distance, and performs a controlled turn and forward adjustment to safely avoid the obstacle.

Surveillance and Alert system:

When the robot senses an obstacle in its critical zone, it will cause the robot to capture an image using its onboard (pi camera) camera and sends a remote alert via telegram with the image attached and distance, a cooldown mechanism is implemented to prevent a repeated spam of alerts within a short time period, which ensures a stable and reliable system.

System Operation:

The system operates in a continuous loop, processing a sensor input, executing movement decisions, and applying a short delay to maintain the real time response time, when the robot is stopped, all motors are safely stopped and the servo is centered to ensure a proper shutdown.

These control algorithms are what enables this autonomous operation, real time decision making and reliable surveillance behavior.

Testing and Results

Comprehensive testing was conducted to evaluate the performance, reliability, and functionality of the autonomous obstacle-avoiding surveillance robot. The tests focused on distance measurement accuracy, obstacle avoidance behavior, alert responsiveness, and system stability.

Test Environment and Setup

All tests were performed in an indoor controlled environment on a flat surface. Obstacles of different sizes and materials were placed at distances ranging from 20 cm to 200 cm. The Raspberry Pi executed the full Python control program with all sensors, motors, camera module, and Telegram communication enabled.

Distance Measurement Accuracy

The ultrasonic sensor readings were tested at different distances while the robot was stationary and in motion.

Results:

- The sensor provided stable readings within the configured range (0–200 cm).
- Using the median filtering technique, sudden spikes and unstable readings were significantly reduced.
- Compared to raw readings, filtered values were more consistent and prevented unnecessary robot stops.

Overall, the distance measurement system proved reliable for real-time obstacle detection.

Obstacle Detection and Avoidance Performance

When an obstacle was detected within the critical threshold (< 65 cm), the robot immediately stopped and activated the avoidance sequence.

Observed behavior:

- The servo successfully scanned the environment from -90° to $+90^\circ$.
- The robot consistently selected the direction with the greatest available distance.
- Pulse-based turning allowed smooth and controlled rotation.
- After avoidance, the robot resumed forward motion without manual intervention.

The robot successfully avoided obstacles in most test cases, demonstrating effective autonomous navigation.

Speed Control and Motion Stability

Different distance zones were tested:

- Far zone (> 145 cm): normal cruising motion
- Slow zone (65–145 cm): reduced speed
- Critical zone (< 65 cm): stop and avoidance

The soft-start mechanism reduced sudden motor jerks at startup, and pulse-based PWM control improved low-speed stability and turning precision.

Alerting and Camera Performance

When an obstacle entered the critical zone:

- The RGB LED changed color
- The buzzer sounded briefly
- A photo was captured and sent via Telegram

Results:

- Telegram alerts were received reliably with minimal delay.
- The cooldown mechanism prevented repeated message spamming.
- When the camera was unavailable, the system safely sent a text alert instead.

System Stability

The robot was tested for extended operation periods.

Results:

- No system crashes or unexpected behavior were observed.
- Motors stopped safely and the servo centered correctly upon program termination.
- Overall operation remained stable and responsive.

Lessons Learned

Sensor Data Handling

The ultrasonic sensor readings were sometimes noisy or inconsistent. Using a median filter helped remove spikes and improved the robot's ability to make reliable decisions about its surroundings.

Motor Control Approach

Direct motor activation caused jerky movements and unstable turns. Switching to pulse-based PWM control with a gradual start made the robot's motion smoother and more precise while reducing mechanical strain.

Benefits of a Movable Sensor

Mounting the ultrasonic sensor on a servo allowed it to scan different angles rather than just forward. This made the robot better at detecting obstacles and choosing safer paths.

Software Organization

Breaking the program into separate functions for movement, sensing, scanning, and alerts made debugging simpler and the code easier to read. This structure also made it easier to test individual parts and plan future improvements.

Reliable Communication

Integrating Telegram messages highlighted the importance of handling errors and adding cooldowns. These measures prevented message flooding and ensured that alerts worked even if network or camera issues occurred.

Importance of Real-World Testing

Testing the robot in a real environment helped refine thresholds, motor timings, and scan delays. Adjusting these parameters through trial and error was key to achieving consistent and reliable performance.

Overall Experience

This project gave practical experience in robotics, sensor integration, embedded programming, and remote monitoring. The lessons learned provide a solid foundation for developing more advanced autonomous robots.

Future Enhancements

Future versions could enhance the robot to overcome its current limitations (navigation precision, the range of its sensor and the power efficiency), while building on the existing framework.

Possible hardware enhancements:

- Integrate the Arduino Uno for low level jobs like sensor readings and motor using serial communication between it and the raspberry pi.
- An IMU could be added for tilt correction and heading estimation sensor.
- The HC-SR04 and servo could be replaced with a better sensor which is called rplidar ai which can provide a 360 degree scanning and an extending range

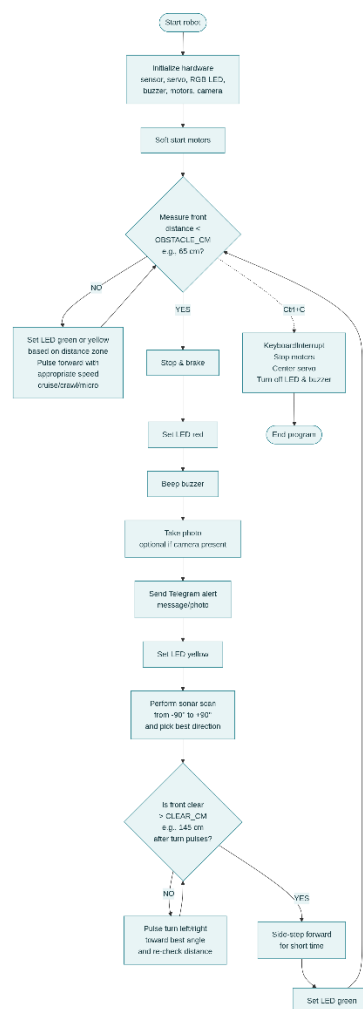
- Use a higher batter pack with battery monitoring for improved power management

Possible software enhancements:

- Integrate OpenCV to detect people and objects in captured images
- The use of yolov5-tiny library for obstacle real time classification
- The adaptation of ROS2 for modular sensor, motor, and vision nodes

Attach diagrams, flowcharts, and photos

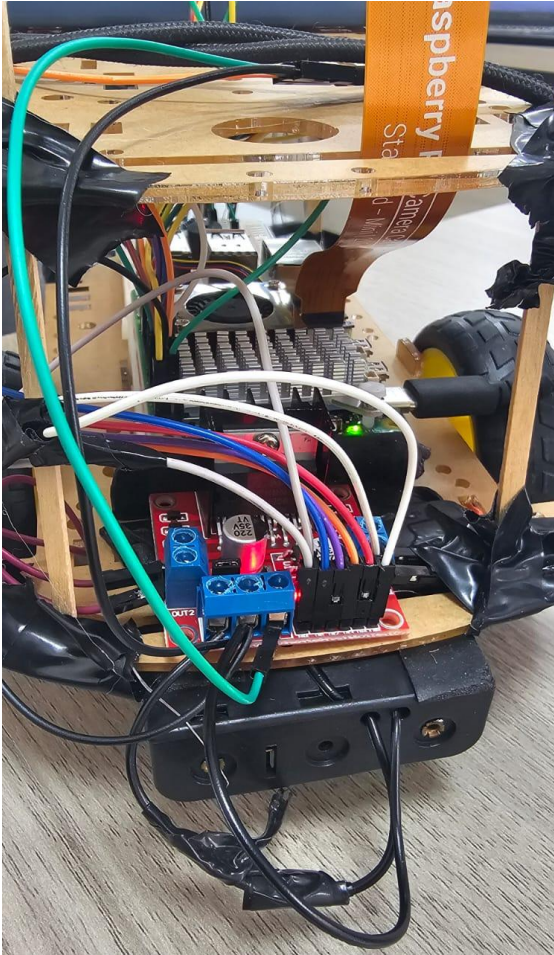
Flowchart:

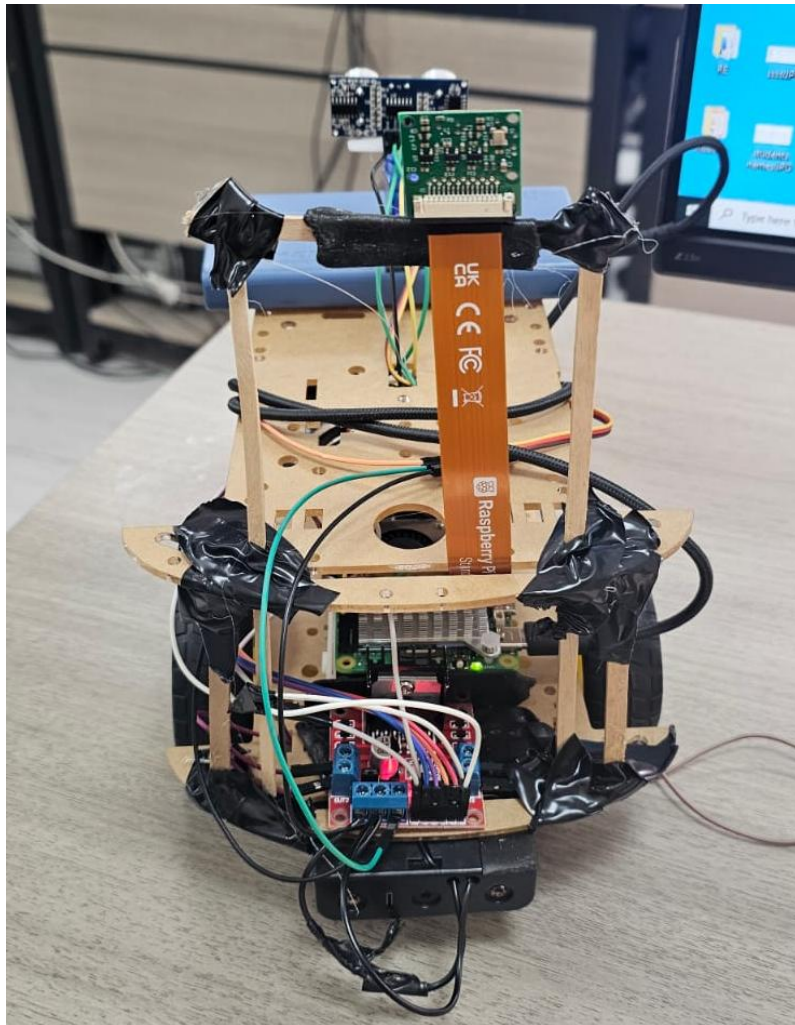


Photos of the:

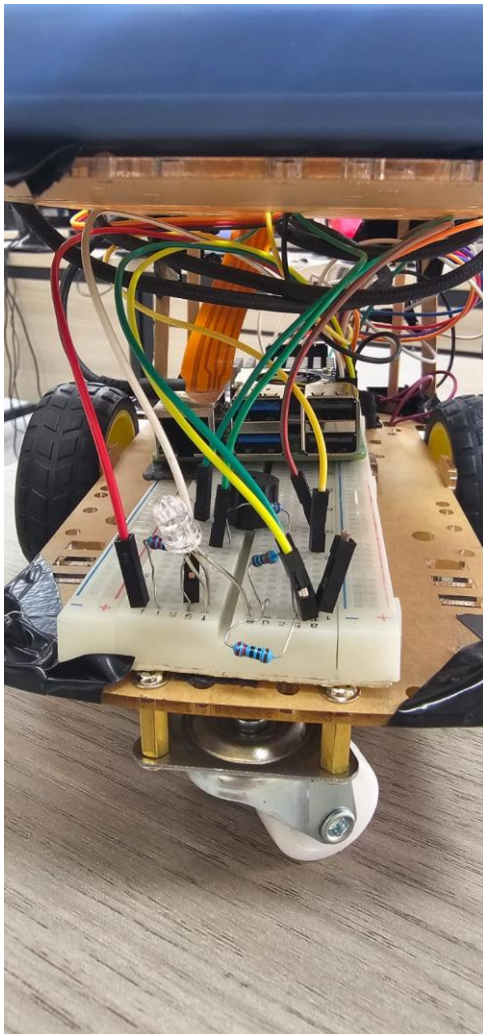
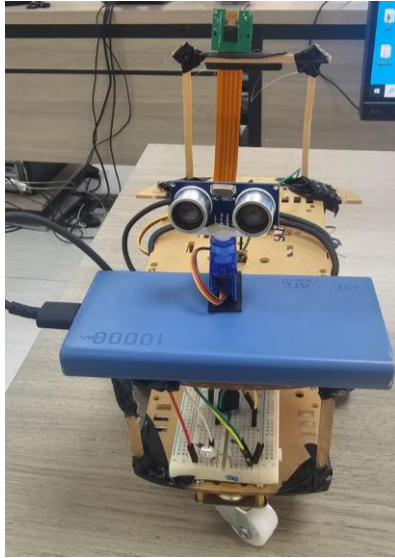
- 1) Robot:

From the backside:

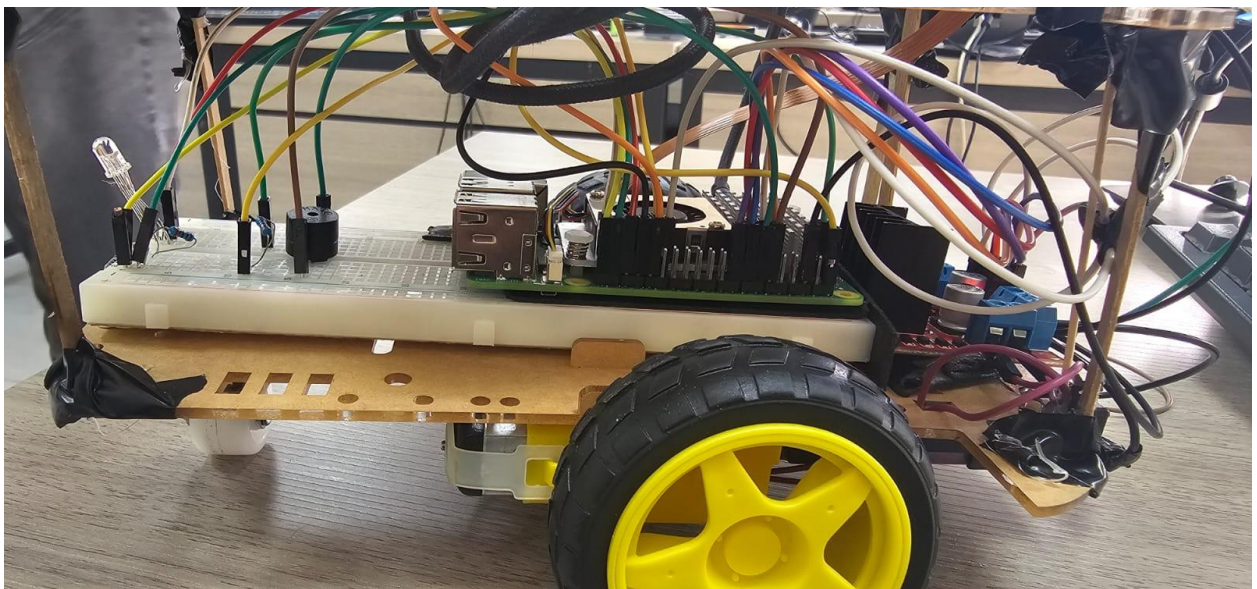
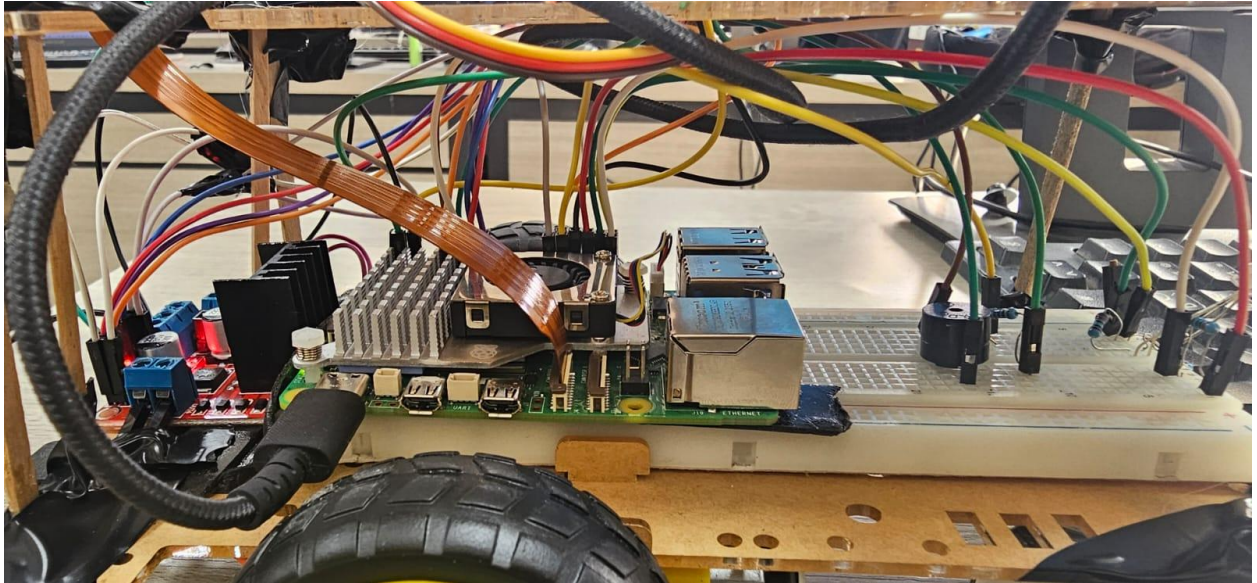


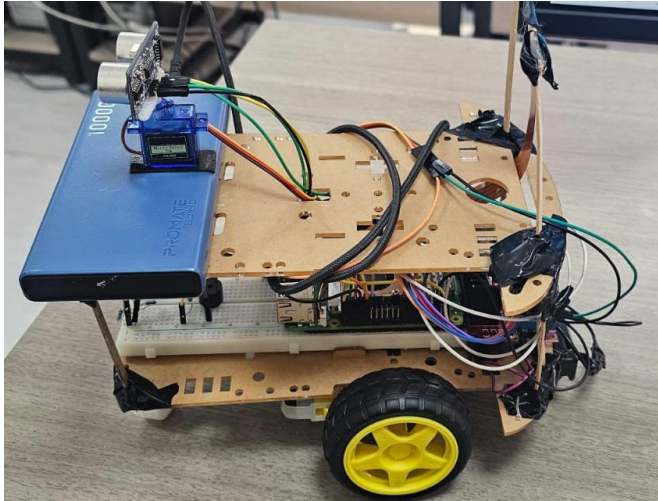


From the frontside:



From the sides:





2) Telegram bot:

